Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №6

по дисциплине «Серверное программирование»

Тема: REST-сервис, транзакции и веб-приложение

Преподаватель _____ _____ _____
подпись, дата | инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – реализовать REST API, использовать слой доступа к данным из практического задания № 5 и обеспечить целостность данных транзакциями. Эталонная реализация – Spring Boot; работа выполнена на Node.js, TypeScript и веб-фреймворке Express (альтернативный стек согласован с преподавателем).

Требования задания:

– REST-сервис с эндпоинтами GET (список), POST (создание) и DELETE (по идентификатору);

– ответы в формате JSON; модель из нескольких таблиц (DTO или проекция);

– сервисный слой с транзакцией для составной бизнес-операции;

– демонстрация отката (rollback) при ошибке в транзакции;

– демо-клиент, отображающий результат GET (необязательно).

Критерии приёмки: приложение запускается; GET, POST и DELETE работают через сервисный слой; JSON и DTO корректны; транзакция: успех фиксируется, при ошибке – откат; демо-клиент. Отчёт содержит URL эндпоинтов, примеры запросов, код сервиса с транзакцией, снимок вывода с откатом и описание уровня изоляции PostgreSQL по умолчанию.

2 Соответствие эталонного стека и использованного

Таблица 1 – Соответствие Java (Spring Boot) и TypeScript (Express)

Эталон (Java)	**Реализация (TypeScript)**
Spring Boot, spring-boot-starter-web	Express 5 (веб-сервер и маршрутизация)
@SpringBootApplication, SpringApplication.run	src/Application.ts: app.listen(8080)
@RestController, @GetMapping, @PostMapping, @DeleteMapping	обработчики app.get, app.post и app.delete в src/app.ts
@RequestBody, JSON	express.json() и res.json()
ResponseEntity.noContent()	res.status(204).end()
DTO (record)	интерфейс RentalDto

Эталон (Java)	Реализация (TypeScript)
@Service	классы StationService, ClientService, RentalService в src/services.ts
@Transactional, @Transactional(readOnly = true)	метод DAO.transactional(fn, { readOnly })
@ExceptionHandler(EntityNotFoundException) → 404	обработчик ошибок Express: NotFoundError → 404
application.properties	src/config.ts
HTML-страница с fetch	public/index.html

Слой данных тот же, что в практических заданиях №4 и №5: ORM Drizzle, пул соединений pg, база данных «ВелоГрад» в PostgreSQL. Сущности описаны в файле model.ts.

3 Запуск приложения

Точка входа – файл Application.ts: приложение слушает порт 8080 и печатает сообщение о запуске (в эталоне – «Started Application on port 8080»). Запуск: npm run pg:start (сервер базы данных), затем npm start.

```
import { app } from './app.ts';

app.listen(8080, () => console.log('Started Application on port 8080'));
```

```
PS ...\pz6-rest-transactions> npm start
Started Application on port 8080
```

Рисунок 1 – Запуск приложения

4 REST-эндпоинты

Эндпоинты приведены в таблице 2; все ответы с телом – в формате JSON. Если запись не найдена, возвращается 404; непредвиденная ошибка – 500 с описанием.

Таблица 2 – Эндпоинты

Метод	URL	Назначение	Ответ
GET	/api/stations	список станций	200
GET	/api/stations/{id}	станция по идентификатору	200 или 404

Метод	URL	Назначение	Ответ
POST	/api/stations	создание станции (JSON в теле)	200
DELETE	/api/stations/{id}	удаление станции	204 или 404
GET	/api/rentals?limit=N	последние поездки: DTO из четырёх таблиц	200
GET	/api/clients/{id}	клиент по идентификатору	200 или 404
POST	/api/clients	регистрация клиента, его кошелька и избранных станций одной транзакцией; параметр fail=true вызывает ошибку после сохранения	200 или 500
DELETE	/api/clients/{id}	удаление клиента	204 или 404
GET	/	демо-клиент (HTML-страница)	200

Файл app.ts содержит все маршруты. Функция route оборачивает обработчик в DAO.run: для каждого запроса создаётся свой контекст сессии (аналог сессии потока). Обработчик ошибок в конце преобразует NotFoundError в ответ 404.

```
import { join } from 'node:path';
import express, { type NextFunction, type Request, type RequestHandler, type Response } from
'express';
import { DAO } from './DAO.ts';
import { ClientService, NotFoundError, RentalService, StationService } from './services.ts';

const stations = new StationService();
const clients = new ClientService();
const rentals = new RentalService();

function route(handler: (req: Request, res: Response) => Promise<void>): RequestHandler {
    return (req, res) => DAO.run(() => handler(req, res));
}

export const app = express();
app.use(express.json());
app.use(express.static(join(import.meta.dirname, '..', 'public')));

app.get('/api/stations', route(async (req, res) => {
    res.json(await stations.list());
}));

app.get('/api/stations/:id', route(async (req, res) => {
    res.json(await stations.get(Number(req.params['id'])));
}));

app.post('/api/stations', route(async (req, res) => {
    res.json(await stations.create(req.body));
}));

app.delete('/api/stations/:id', route(async (req, res) => {
    await stations.delete(Number(req.params['id']));
    res.status(204).end();
}));

app.get('/api/rentals', route(async (req, res) => {
    res.json(await rentals.list(Number(req.query['limit'] ?? 10)));
}));
```

```
app.get('/api/clients/:id', route(async (req, res) => {
    res.json(await clients.get(Number(req.params['id'])));
}));

app.post('/api/clients', route(async (req, res) => {
    const { favoriteStationIds = [], ...data } = req.body;
    res.json(await clients.register(data, favoriteStationIds, req.query['fail'] === 'true'));
}));

app.delete('/api/clients/:id', route(async (req, res) => {
    await clients.delete(Number(req.params['id']));
    res.status(204).end();
}));

app.use((error: unknown, req: Request, res: Response, next: NextFunction) => {
    if (error instanceof NotFoundError) {
        res.status(404).json({ error: error.message });
        return;
    }
    res.status(500).json({ error: error instanceof Error ? error.message : String(error) });
});
```

5 Примеры запросов

Запросы выполнены утилитой curl к работающему приложению. Тела запросов POST лежат в файлах JSON (кодировка UTF-8):

```
station.json
{"districtId":1,"name":"Новая станция","address":"ул. Ленина,
1","capacity":10,"latitude":"55.75","longitude":"37.60","openedOn":"2026-11-05"}

client.json
{"lastName":"Тестов","firstName":"Тест","phone":"+79990000001","birthDate":"2000-01-01","favoriteStationIds":
[1,2]}
```

Получение станции и списка поездок (DTO из четырёх таблиц: rental, client, station, tariff) – рисунок 2.

```
$ curl http://localhost:8080/api/stations/1
{"id":1,"districtId":1,"name":"Площадь Революции","address":"ул. Тверская,
1","capacity":24,"latitude":"55.755800","longitude":"37.617600","openedOn":"2021-05-14"}
HTTP 200

$ curl "http://localhost:8080/api/rentals?limit=2"
[{"rentalId":400000,"client":"Иванов Михаил","startStation":"Станция №279","tariff":"MIN_NIGHT","startedAt":"2026-10-27
05:55:32","totalCost":"292.00"},{"rentalId":10016,"client":"Беляева Ксения","startStation":"Станция
№56","tariff":"PROMO_1","startedAt":"2026-10-27 05:52:49","totalCost":"0.00"}]
HTTP 200
```

Рисунок 2 – GET /api/stations/1 и GET /api/rentals?limit=2

Создание и удаление станции – рисунок 3: POST вернул созданную запись с идентификатором, DELETE – код 204 без тела.

```
$ curl -X POST http://localhost:8080/api/stations -H "Content-Type: application/json" -d @station.json
{"id":305,"districtId":1,"name":"Новая станция","address":"ул. Ленина,
1","capacity":10,"latitude":"55.750000","longitude":"37.600000","openedOn":"2026-11-05"}
HTTP 200

$ curl -X DELETE http://localhost:8080/api/stations/305
HTTP 204
```

Рисунок 3 – POST и DELETE станции

Запрос несуществующей станции – рисунок 4: обработчик ошибок возвращает 404 и описание.

```
$ curl http://localhost:8080/api/stations/999999
{"error":"station 999999 not found"}
HTTP 404
```

Рисунок 4 – Ответ 404

6 Составная модель (DTO)

Метод GET /api/rentals возвращает данные из нескольких таблиц одним JSON-объектом: поездка, фамилия и имя клиента, станция начала, код тарифа, время начала и стоимость. Для этого сервис RentalService соединяет четыре таблицы (rental, client, station, tariff) и формирует объекты RentalDto (рисунок 2). Код сервиса приведён в разделе 7.

7 Сервисный слой и транзакции

Бизнес-логика вынесена из обработчиков в сервисы (файл services.ts). Транзакцию (аналог @Transactional с распространением REQUIRED) выполняет метод DAO.transactional: если транзакция уже открыта – метод выполняется внутри неё; иначе выполняются BEGIN, затем функция, затем COMMIT; при любом исключении выполняется ROLLBACK, и исключение передаётся дальше. Параметр readOnly = true открывает транзакцию BEGIN READ ONLY – так выполняются методы чтения (аналог @Transactional(readOnly = true)).

```
import { AsyncLocalStorage } from 'node:async_hooks';
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool, type PoolClient } from 'pg';
import { config } from './config.ts';

function open(client: PoolClient) {
    return drizzle(client);
}
```

```
export type Session = ReturnType<typeof open>;

interface Scope {
    session: Session | null;
    inTransaction: boolean;
}

export class DAO {
    static sessionFactory = new Pool(config);
    static storage = new AsyncLocalStorage<Scope>();

    static async run<T>(fn: () => Promise<T>): Promise<T> {
        return DAO.storage.run({ session: null, inTransaction: false }, async () => {
            try {
                return await fn();
            } finally {
                DAO.scope().session?.$client.release();
            }
        });
    }

    static scope(): Scope {
        const scope = DAO.storage.getStore();
        if (scope === undefined) throw new Error('DAO.run() is required');
        return scope;
    }

    static async getSession(): Promise<Session> {
        const scope = DAO.scope();
        if (scope.session === null) scope.session = open(await DAO.sessionFactory.connect());
        return scope.session;
    }

    static async begin(readOnly = false): Promise<void> {
        const session = await DAO.getSession();
        await session.$client.query(readOnly ? 'BEGIN READ ONLY' : 'BEGIN');
        DAO.scope().inTransaction = true;
    }

    static async commit(): Promise<void> {
        const session = await DAO.getSession();
        await session.$client.query('COMMIT');
        DAO.scope().inTransaction = false;
    }

    static async rollback(): Promise<void> {
        const session = await DAO.getSession();
        await session.$client.query('ROLLBACK');
        DAO.scope().inTransaction = false;
    }

    static async transactional<T>(fn: () => Promise<T>, options: { readOnly?: boolean } = {}): Promise<T>
{
        if (DAO.scope().inTransaction) return fn();
        await DAO.begin(options.readOnly ?? false);
        try {
            const result = await fn();
            await DAO.commit();
            return result;
        } catch (error) {
            await DAO.rollback();
            throw error;
        }
    }

    static async shutdown(): Promise<void> {
        await DAO.sessionFactory.end();
    }
}
```

Составная бизнес-операция – метод ClientService.register: в одной транзакции сохраняются клиент, его кошелёк и избранные станции; если задан признак failAfterSave, после сохранения выбрасывается исключение, и все три записи должны откатиться. Остальные методы читают данные в транзакциях только для чтения или изменяют их в обычных транзакциях.

```
import { desc, eq, sql } from 'drizzle-orm';
import { DAO } from './DAO.ts';
import { client, clientFavoriteStation, clientWallet, type NewClient, type NewStation, rental, station, type
Station, tariff } from './model.ts';

export class NotFoundError extends Error {}

export interface RentalDto {
    rentalId: number;
    client: string;
    startStation: string;
    tariff: string;
    startedAt: string;
    totalCost: string | null;
}

export class StationService {
    async list(): Promise<Station[]> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            return session.select().from(station).orderBy(station.id);
        }, { readOnly: true });
    }

    async get(id: number): Promise<Station> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const rows = await session.select().from(station).where(eq(station.id, id));
            if (rows[0] === undefined) throw new NotFoundError(`station ${id} not found`);
            return rows[0];
        }, { readOnly: true });
    }

    async create(data: NewStation): Promise<Station> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const rows = await session.insert(station).values(data).returning();
            return rows[0]!;
        });
    }

    async delete(id: number): Promise<void> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const result = await session.delete(station).where(eq(station.id, id));
            if (result.rowCount === 0) throw new NotFoundError(`station ${id} not found`);
        });
    }
}

export class ClientService {
    async get(id: number): Promise<typeof client.$inferSelect> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const rows = await session.select().from(client).where(eq(client.id, id));
            if (rows[0] === undefined) throw new NotFoundError(`client ${id} not found`);
            return rows[0];
        }, { readOnly: true });
    }

    async register(data: NewClient, favoriteStationIds: number[], failAfterSave: boolean): Promise<typeof
client.$inferSelect> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const created = (await session.insert(client).values(data).returning())[0]!;
            await session.insert(clientWallet).values({ clientId: created.id });
```

```
            if (favoriteStationIds.length > 0) {
                await session.insert(clientFavoriteStation).values(favoriteStationIds.map((stationId) => ({
clientId: created.id, stationId })));
            }
            if (failAfterSave) throw new Error('test rollback');
            return created;
        });
    }

    async delete(id: number): Promise<void> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            const result = await session.delete(client).where(eq(client.id, id));
            if (result.rowCount === 0) throw new NotFoundError(`client ${id} not found`);
        });
    }
}

export class RentalService {
    async list(limit: number): Promise<RentalDto[]> {
        return DAO.transactional(async () => {
            const session = await DAO.getSession();
            return session
                .select({
                    rentalId: rental.id,
                    client: sql<string>`${client.lastName} || ' ' || ${client.firstName}`,
                    startStation: station.name,
                    tariff: tariff.code,
                    startedAt: rental.startedAt,
                    totalCost: rental.totalCost,
                })
                .from(rental)
                .innerJoin(client, eq(client.id, rental.clientId))
                .innerJoin(station, eq(station.id, rental.startStationId))
                .innerJoin(tariff, eq(tariff.id, rental.tariffId))
                .orderBy(desc(rental.startedAt))
                .limit(limit);
        }, { readOnly: true });
    }
}
```

8 Демонстрация отката транзакции

Сценарий: клиент с избранными станциями регистрируется запросом POST /api/clients?fail=true – после сохранения всех записей сервис выбрасывает исключение. Затем тот же запрос выполняется без ошибки. Состояние базы данных проверяется отдельным соединением psql: в приведённых запросах подсчитываются клиент, его кошелёк и его избранные станции.

```
$ curl -X POST "http://localhost:8080/api/clients?fail=true" -H "Content-Type: application/json" -d @client.json
{"error":"test rollback"}
HTTP 500

$ psql -d velograd -c "SELECT (SELECT count(*) FROM client WHERE phone = '+79990000001') AS clients, (SELECT count(*) FROM
client_wallet w JOIN client c ON c.id = w.client_id WHERE c.phone = '+79990000001') AS wallets, (SELECT count(*) FROM
client_favorite_station f JOIN client c ON c.id = f.client_id WHERE c.phone = '+79990000001') AS favorites"
 clients | wallets | favorites
---------+---------+-----------
       0 |       0 |         0
(1 row)
```

Рисунок 5 – Запрос с ошибкой: ответ 500, в базе данных нет ни клиента, ни кошелька, ни избранных станций (откат)

После ошибки в базе нет ни одной из трёх записей: откат отменил все изменения транзакции. Запрос без ошибки (рисунок 6) сохранил клиента, кошелёк и две избранные станции; затем тестовый клиент удалён запросом DELETE (каскадное удаление убирает кошелёк и избранное).

```
$ curl -X POST http://localhost:8080/api/clients -H "Content-Type: application/json" -d @client.json
{"id":50006,"lastName":"Тестов","firstName":"Тест","phone":"+79990000001","email":null,"birthDate":"2000-01-01"}
HTTP 200

$ psql -d velograd -c "SELECT (SELECT count(*) FROM client WHERE phone = '+79990000001') AS clients, (SELECT count(*) FROM
client_wallet w JOIN client c ON c.id = w.client_id WHERE c.phone = '+79990000001') AS wallets, (SELECT count(*) FROM
client_favorite_station f JOIN client c ON c.id = f.client_id WHERE c.phone = '+79990000001') AS favorites"
 clients | wallets | favorites
---------+---------+-----------
       1 |       1 |         2
(1 row)

$ curl -X DELETE http://localhost:8080/api/clients/50006
HTTP 204
```

Рисунок 6 – Запрос без ошибки: клиент, кошелёк и две избранные станции сохранены

Тот же сценарий выполняет программа demo.ts (npm run demo): она запускает приложение на свободном порту, вызывает API и проверяет базу данных, после чего удаляет созданные данные.

```
import { DAO } from './DAO.ts';
import { app } from './app.ts';

const server = app.listen(0);
const base = `http://127.0.0.1:${(server.address() as { port: number }).port}`;

async function call(method: string, path: string, body?: unknown): Promise<{ status: number; body: unknown }> {
    const response = await fetch(base + path, {
        method,
        headers: { 'Content-Type': 'application/json' },
        ...(body === undefined ? {} : { body: JSON.stringify(body) }),
    });
    const text = await response.text();
    return { status: response.status, body: text === '' ? null : JSON.parse(text) };
}

const phone = `+7999${String(Date.now()).slice(-7)}`;
const client = { lastName: 'Тестов', firstName: 'Тест', phone, birthDate: '2000-01-01', favoriteStationIds: [1,
2] };

console.log('GET /api/stations ->', (await call('GET', '/api/stations')).status);
console.log('GET /api/rentals?limit=2 ->', JSON.stringify((await call('GET', '/api/rentals?limit=2')).body));
console.log('GET /api/stations/999999 ->', (await call('GET', '/api/stations/999999')).status);

const failed = await call('POST', '/api/clients?fail=true', client);
console.log('POST /api/clients?fail=true ->', failed.status, JSON.stringify(failed.body));
console.log('clients with this phone after the error:', await DAO.run(async () => {
    const session = await DAO.getSession();
    const result = await session.$client.query('SELECT count(*)::int AS n FROM client WHERE phone = $1',
[phone]);
    return result.rows[0].n;
}));

const created = await call('POST', '/api/clients', client);
console.log('POST /api/clients ->', created.status, JSON.stringify(created.body));
const id = (created.body as { id: number }).id;
console.log(`GET /api/clients/${id} ->`, (await call('GET', `/api/clients/${id}`)).status);
console.log(`DELETE /api/clients/${id} ->`, (await call('DELETE', `/api/clients/${id}`)).status);

server.close();
await DAO.shutdown();
```

```
PS ...\pz6-rest-transactions> npm run demo
GET /api/stations -> 200
GET /api/rentals?limit=2 -> [{"rentalId":400000,"client":"Иванов Михаил","startStation":"Станция
№279","tariff":"MIN_NIGHT","startedAt":"2026-10-27 05:55:32","totalCost":"292.00"},{"rentalId":10016,"client":"Беляева
Ксения","startStation":"Станция №56","tariff":"PROMO_1","startedAt":"2026-10-27 05:52:49","totalCost":"0.00"}]
GET /api/stations/999999 -> 404
POST /api/clients?fail=true -> 500 {"error":"test rollback"}
clients with this phone after the error: 0
POST /api/clients -> 200 {"id":50002,"lastName":"Тестов","firstName":"Тест","phone":"+79991242534","email":null,"birthDate":"2000-
01-01"}
GET /api/clients/50002 -> 200
DELETE /api/clients/50002 -> 204
```

Рисунок 7 – Вывод программы demo.ts

9 Уровень изоляции транзакций PostgreSQL по умолчанию

По умолчанию PostgreSQL использует уровень изоляции READ COMMITTED (рисунок 8). На этом уровне каждая команда видит только данные, зафиксированные до её начала: «грязное» чтение (чтение незафиксированных изменений других транзакций) невозможно, но результаты двух одинаковых запросов внутри одной транзакции могут различаться, если

другая транзакция зафиксировала изменения между ними (неповторяемое чтение). Для приложения «ВелоГрад» этого достаточно: составные операции атомарны (всё или ничего), а более строгие уровни (REPEATABLE READ, SERIALIZABLE) нужны для расчётов, требующих одинакового снимка данных на всём протяжении транзакции.

```
psql -d velograd -c "SHOW default_transaction_isolation"
 default_transaction_isolation
-------------------------------
 read committed
(1 row)
```

Рисунок 8 – Уровень изоляции по умолчанию

10 Демо-клиент

Демо-клиент – HTML-страница public/index.html, которую приложение отдаёт по адресу «/»: страница запрашивает fetch('/api/stations') и выводит ответ в таблицу (рисунок 9).

```
<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<title>Станции</title>
</head>
<body>
<table border="1">
<thead><tr><th>ID</th><th>Название</th><th>Адрес</th><th>Мест</th></tr></thead>
<tbody id="rows"></tbody>
</table>
<script>
fetch('/api/stations')
    .then((response) => response.json())
    .then((stations) => {
        document.getElementById('rows').innerHTML = stations
            .map((s) => `<tr><td>${s.id}</td><td>${s.name}</td><td>${s.address}</td><td>${s.capacity}</td>
</tr>`)
            .join('');
    });
</script>
</body>
</html>
```

ID	Название	Адрес	Мест
1	Площадь Революции	ул. Тверская, 1	24
2	Городской сад	Садовая ул., 12	18
3	Морской вокзал	Портовая ул., 3	30
4	Приморский бульвар	Приморский б-р, 7	20
5	Заречье	Набережная ул., 44	16
6	Октябрьская	пр. Октября, 21	26
7	Университет	Университетская пл., 1	40
8	Студгородок	ул. Академика Павлова, 9	34
9	Советская	Советская ул., 58	14
10	Парк Победы	Парковая аллея, 2	22
11	Московские ворота	Московский пр., 105	28
12	Северное депо	Северный проезд, 16	12
13	Станция №13	наб. Речная, 2	15
14	Станция №14	ул. Ленина, 84	22
15	Станция №15	ул. Пушкина, 66	29
16	Станция №16	ул. Гагарина, 108	30

Рисунок 9 – Демо-клиент в браузере (снимок страницы http://localhost:8080/)

11 Выводы

1 Реализовано REST-приложение на Express с эндпоинтами GET, POST и DELETE для станций и клиентов; ответы – JSON, отсутствующая запись даёт 404, ошибка – 500.

2 Метод GET /api/rentals возвращает составную модель (DTO) из четырёх таблиц одним JSON-объектом.

3 Бизнес-логика вынесена в сервисный слой; транзакции реализованы методом DAO.transactional: фиксация при успехе, откат при ошибке, транзакции только для чтения для запросов GET.

4 Откат продемонстрирован: при ошибке после сохранения клиента, кошелька и избранных станций в базе данных не осталось ни одной из записей; тот же запрос без ошибки сохранил все три.

5 Уровень изоляции PostgreSQL по умолчанию – READ COMMITTED; демо-клиент отображает результат GET /api/stations в таблице.